

Pflichtenheft

Film- & Serien-Tracker

Webbasierte Anwendung zur Verwaltung persönlicher Film- und Serienaktivitäten

Projektbezeichnung	Film- & Serien-Tracker
Modul	WI-PRO1 – Projekt: Software Engineering (2. Semester)
Studiengang / Gruppe	Wirtschaftsinformatik, WI25 – DHSN Glauchau
Team	Team 2 – Simon Heckel, Matteo Kluge, Fabian Katzung
Dokumenttyp	Pflichtenheft (Auftragnehmer-Antwort auf das Lastenheft v1.4)
Bezug Lastenheft	Lastenheft v1.4 vom 19.05.2026
Version	1.1
Erstellt am	26.05.2026
Status	Vorläufig abgenommen – Fähig zur Weiterentwicklung

Inhaltsverzeichnis

Vorbemerkung.....	4
1 Zielbestimmung	4
1.1 Muss-Kriterien	4
1.2 Soll-Kriterien	4
1.3 Kann-Kriterien.....	4
1.4 Abgrenzungskriterien (bewusste Ausschlüsse)	5
2 Produkteinsatz.....	5
2.1 Anwendungsbereiche	5
2.2 Zielgruppen.....	5
2.3 Betriebsbedingungen.....	5
3 Produktübersicht	6
4 Produktfunktionen.....	7
4.1 Nutzerverwaltung.....	7
4.2 Filmkatalog	7
4.3 Serienkatalog	8
4.4 Lokale Nutzerlisten	8
4.5 Listen-Features	8
4.6 Suche und Filter	9
4.7 Optionale Funktionen.....	9
4.8 Risikofaktoren.....	10
5 Produktdaten.....	10
5.1 Entity-Relationship-Modell.....	10
5.2 Relationsschema.....	11
5.3 Konkretisierung der Datentypen	11
6 Produktleistungen (nicht-funktionale Anforderungen).....	12
7 Qualitätsanforderungen	12
8 Benutzungsoberfläche (GUI).....	13
9 Technische Produktumgebung	14
9.1 Architekturansatz	14
9.2 Empfohlener Technologiestack	15
9.3 Externe Schnittstelle TMDb	15
10 Programmabläufe (PAP)	16
10.1 Anwendungsfall: Nutzer fragt einen Film an	16
10.2 Anwendungsfall: Nutzer ruft seine Liste auf	17
11 Iterationsplan und Aufwandsschätzung	18
11.1 Rahmen.....	18
11.2 Risikofaktoren.....	18
11.3 Sprintplan	18

11.4 Aufgabenverteilung	19
12 Abnahmekriterien.....	19
13 Anhänge und mitgeltende Dokumente	19
14 Glossar	20

Vorbemerkung

Dieses Pflichtenheft beschreibt, WIE die im Lastenheft v1.4 spezifizierten Anforderungen umgesetzt werden. Das Lastenheft beantwortet die Frage „Was soll die Anwendung leisten?“, das vorliegende Pflichtenheft beantwortet die Frage „Wie wird das konkret realisiert?“. Die Anforderungs-IDs aus dem Lastenheft (FA-01 ff., NFA-01 ff., OA-01 ff.) werden übernommen und um konkrete Lösungsansätze, Akzeptanzkriterien und Verweise auf Architektur, Datenmodell und Programmabläufe ergänzt.

Personelle Rahmenbedingung: Das Team besteht aus zwei Studierenden mit unterschiedlichem Programmiererfahrungsstand (eine Person Einsteiger, eine Person mittlere Erfahrung). Das verfügbare Zeitbudget umfasst 75 Lehrveranstaltungen à 45 Minuten ($\approx$ 56 Stunden Präsenz- bzw. veranschlagte Projektzeit pro Person). Daraus ergibt sich ein bewusst konservativ geschnittener Funktionsumfang in der ersten Iteration: Alle Muss-Anforderungen werden vollständig realisiert, ausgewählte Soll- und Kann-Anforderungen je nach Restbudget. Optionale Wunschfunktionen werden nur dann angegangen, wenn der Kernumfang stabil läuft.

1 Zielbestimmung

1.1 Muss-Kriterien

Die Anwendung muss alle folgenden Punkte erfüllen, damit das Projekt als erfolgreich abgeschlossen gilt:

- Registrierung und Anmeldung mit Benutzername und sicher gespeichertem Passwort (Hashing, kein Klartext).
- Strikte Datentrennung zwischen Nutzern: Jeder Nutzer sieht ausschließlich seine eigenen Listen.
- Zentrale Schnittstelle auf eine Film- und Seriendatenbank mit den im Lastenheft genannten Pflichtfeldern.
- Persistente „Schon-gesehen-Liste“ pro Nutzer.
- Suchfunktion für Filme und Serien nach Titel.
- Nutzbarkeit ohne Installation über einen aktuellen Standard-Webbrowser.

1.2 Soll-Kriterien

Die folgenden Punkte werden umgesetzt, sofern das Restbudget nach Realisierung der Musskriterien dies zulässt:

- Watchlist mit der Möglichkeit, Einträge zwischen den Listen zu verschieben.
- Lokale, nutzerspezifische Liste für Einträge, die nicht im zentralen Katalog gefunden werden.
- Komfortable, ansprechende grafische Oberfläche („GUI“ – siehe Abschnitt 8) mit Postern/Thumbnails (OA-05).
- Antwortzeit < 2 Sekunden unter normaler Last (max. 5 gleichzeitige Nutzer in der Abnahmedemo)

1.3 Kann-Kriterien

- Schau-ich-gerade-Liste mit der Möglichkeit, Einträge zwischen den Listen zu verschieben.
- Vereinfachte Such-Vorschläge (Auto-Vervollständigung) bei der Titelsuche.
- Filter nach Genre, Erscheinungsjahr und Laufzeit.
- Passwort-vergessen-Funktion über Sicherheitsfragen anbieten.

1.4 Abgrenzungskriterien (bewusste Ausschlüsse)

- Nutzer können keine neuen Einträge zur zentralen Datenbank einreichen (Skalierungs- und Moderationsaufwand).
- Keine Social-Funktionen (Folgen, Kommentieren, Freundeslisten) – nicht Bestandteil der ersten Iteration.
- Keine Bezahl- oder Abo-Funktionen.
- Kein automatisches Periodisches Cron-basiertes Aktualisieren des lokalen Caches – Cache wird ausschließlich passiv durch Nutzeranfragen befüllt.
- Keine produktive Bereitstellung („Deployment“) auf einem öffentlich erreichbaren Server – Demonstration läuft lokal bzw. im Hochschulnetz.

2 Produkteinsatz

2.1 Anwendungsbereiche

Die Anwendung dient der zentralen Verwaltung persönlicher Film- und Serienaktivitäten über mehrere Streaming-Dienste hinweg. Sie ersetzt händische Notizen und Browser-Lesezeichen durch eine strukturierte, nutzerspezifische Datenhaltung mit zentralem Medienkatalog.

2.2 Zielgruppen

Primäre Zielgruppe sind Privatanutzer, die regelmäßig Filme und Serien auf mehreren Streaming-Plattformen konsumieren und ihre Aktivitäten plattformübergreifend nachverfolgen möchten. Es werden keine besonderen IT-Kenntnisse vorausgesetzt; die Bedienung soll sich an gängigen Streaming-Portalen orientieren.

2.3 Betriebsbedingungen

- Verteilte Anwendung: Client (Browser) und Server laufen auf getrennten Systemen.
- Server: lauffähig auf einem handelsüblichen Notebook (Linux/Windows/macOS), für die Demonstration genügt der Entwicklerrechner.
- Client: jeder aktuelle Browser (Chromium-, Firefox-, Safari-basiert) mit aktiviertem JavaScript.
- Internetverbindung erforderlich für TMDb-Anfragen; bei Ausfall greift der Fallback-Mechanismus (siehe Abschnitt 9.1 und PAP).
- Keine besondere Hardware (kein GPU, keine spezielle Mobilhardware) erforderlich.

3 Produktübersicht

Der Film- & Serien-Tracker ist eine verteilte Web-Anwendung. Nutzer greifen über den Browser auf einen lokal betriebenen Server zu, der eine REST-Schnittstelle bereitstellt. Der Server fragt bei TMDb (The Movie Database) Stamm- und Detaildaten ab, speichert sie zur Geschwindigkeits- und Ausfallsicherung in einer lokalen Datenbank zwischen und verwaltet die nutzerspezifischen Listen.

Das nachfolgende Use-Case-Diagramm zeigt die Funktionen aus Sicht des Nutzers und die Integration der externen TMDb-API.

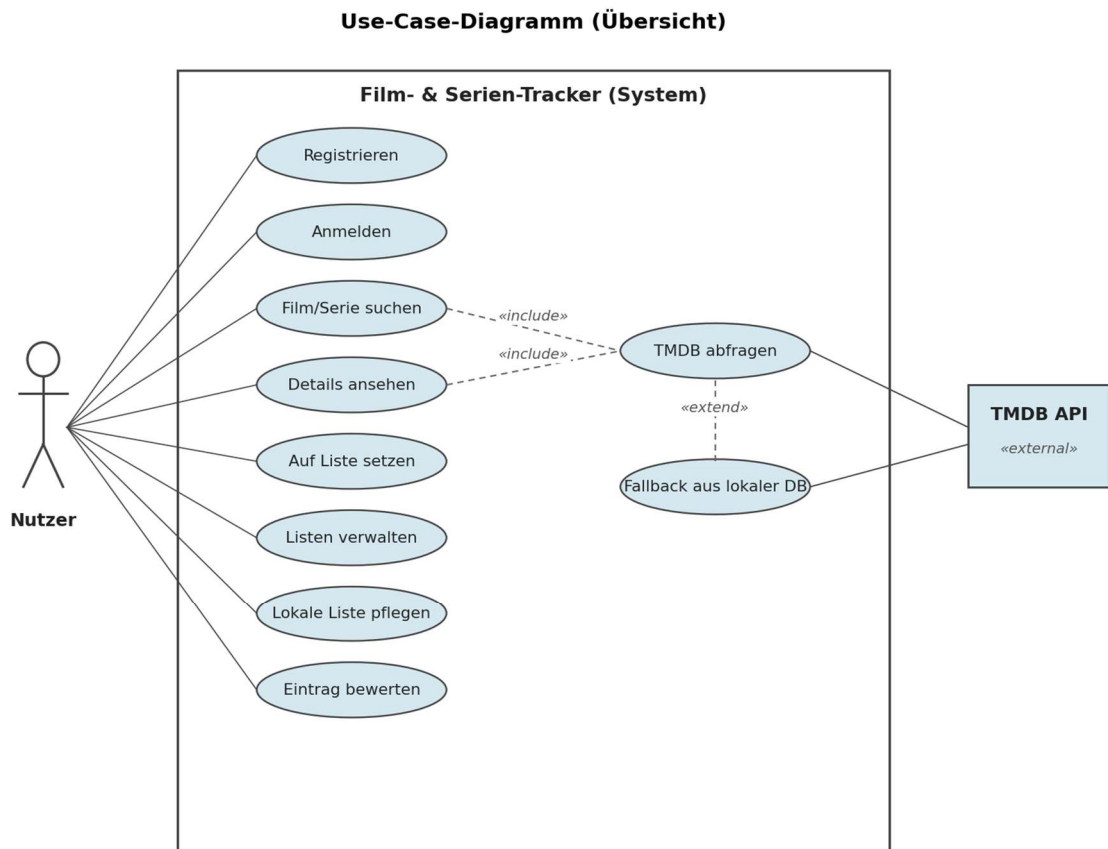


Abb. 1: Use-Case-Diagramm – Funktionen aus Nutzersicht und Anbindung der TMDb-API

4 Produktfunktionen

Die folgenden Tabellen konkretisieren die Anforderungen aus Kapitel 3 des Lastenhefts. Jede Funktion erhält die ursprüngliche ID, eine Beschreibung der konkreten Umsetzung und ein knappes Akzeptanzkriterium.

4.1 Nutzerverwaltung

ID	Umsetzung im Pflichtenheft	Akzeptanzkriterium
FA-01	Registrierungsformular mit Feldern Benutzername (eindeutig, min. 3 Zeichen) und Passwort (min. 8 Zeichen). Validierung im Backend.	Anlegen eines neuen Nutzers funktioniert; doppelter Username wird mit Fehlermeldung abgelehnt.
FA-02	Login-Formular auf Startseite; Session-Verwaltung via signiertem Cookie (Flask-Login bzw. äquivalentes Modul). Logout-Schaltfläche im Header.	Nach erfolgreichem Login werden die persönlichen Listen angezeigt; falsche Zugangsdaten erzeugen Fehlermeldung.
FA-03 (Kann)	Optional: bei Registrierung kann eine Sicherheitsfrage hinterlegt werden. Zurücksetzen erfolgt durch Beantworten und Vergabe eines neuen Passworts.	Bei korrekter Antwort kann ein neues Passwort gesetzt werden; bei falscher Antwort wird der Vorgang abgebrochen.
FA-04	Jede datenbankseitige Abfrage von Nutzerdaten enthält den user_id-Filter aus der Session. Es existiert keine API-Route, über die ein Nutzer fremde Listen abrufen kann.	Manueller Test mit zwei Konten zeigt: Nutzer A sieht ausschließlich seine eigenen Listen.
FA-05	Passwörter werden mit bcrypt (Standardparameter, ≥ 12 Runden) gehasht und gesalzt. Klartextpasswörter werden niemals geloggt oder in der DB gespeichert.	In der Tabelle User steht im Feld password ausschließlich ein bcrypt-Hash (Format \$2b\$...).

4.2 Filmkatalog

ID	Umsetzung im Pflichtenheft	Akzeptanzkriterium
FA-06	Server-seitige REST-Endpunkte /api/movies/search und /api/movies/{id}. Pflichtfelder werden direkt aus dem TMDb-Response gemappt; fehlende Felder werden mit „–“ angezeigt.	Für mindestens 5 bekannte Beispielfilme werden alle Pflichtfelder korrekt zurückgeliefert.
FA-07 (Soll)	Anbindung an die TMDb v3-REST-API mit API-Schlüssel über Header.	Die Anwendung sendet bei jedem API-Request den gültigen TMDb-API-Schlüssel im HTTP-Header, erhält eine erfolgreiche Antwort von der TMDb v3 REST API und verarbeitet diese korrekt. Bei fehlendem oder ungültigem Schlüssel wird eine entsprechende Fehlermeldung ausgegeben.

4.3 Serienkatalog

ID	Umsetzung im Pflichtenheft	Akzeptanzkriterium
FA-08	Analog zu Filmen, eigene Endpunkte /api/series/search und /api/series/{id}. Unterscheidung Film/Serie über das Feld typ in FilmeSerien.	Für mindestens 3 bekannte Serien werden alle Pflichtfelder (inkl. Staffel-/Folgenanzahl) korrekt zurückgeliefert.
FA-09 (Soll)	Gemeinsame TMDb-Anbindung mit Filmkatalog; Endpunkt-Routing nach Medientyp.	Beim Wechsel zwischen Film- und Serienansicht werden korrekte TMDb-Endpunkte aufgerufen.

4.4 Lokale Nutzerlisten

ID	Umsetzung im Pflichtenheft	Akzeptanzkriterium
FA-10 (Kann)	Eintrag eines manuellen Datensatzes ohne tmdb_id über ein Formular (Titel, Typ, optionale Notiz). Speicherung in FilmeSerien mit tmdb_id = NULL, zugeordnet ausschließlich dem anlegenden Nutzer.	Ein manuell angelegter Eintrag erscheint in den eigenen Listen, ist für andere Nutzer aber nicht sichtbar/auffindbar.

4.5 Listen-Features

ID	Umsetzung im Pflichtenheft	Akzeptanzkriterium
FA-11 (Soll)	Watchlist als Wert „watchlist“ im Feld listentyp der Tabelle UserFilmeSerien. UI-Schaltfläche „Auf Watchlist“ bei Filmdetails.	Hinzugefügte Filme erscheinen in der Watchlist-Ansicht des Nutzers.
FA-12	„Schon-gesehen“ als Wert „seen“ in UserFilmeSerien.listentyp. Eigene Übersichtsseite mit chronologischer Sortierung.	Markierte Filme erscheinen in „Schon gesehen“; Status wird über Sessions hinweg beibehalten.
FA-13 (Kann)	„Schaue-ich-aktuell“ als Wert „watching“. Drei Listenansichten teilen sich dieselbe Komponente.	Ein Eintrag kann in „watching“ überführt und dort angezeigt werden.
FA-14 (Kann)	Endpunkt PATCH /api/list/{filmserien_id} ändert listentyp; UI-Button „Verschieben nach ...“ in der Listenansicht.	Ein Eintrag wird aus „Watchlist“ in „Schon gesehen“ verschoben und erscheint danach nur noch dort.

4.6 Suche und Filter

ID	Umsetzung im Pflichtenheft	Akzeptanzkriterium
FA-15	Server-seitige Volltextsuche über TMDb-Suchendpunkt. UI-Suchfeld in der Kopfzeile auf allen Seiten.	Die Suche nach „Inception“ liefert den entsprechenden Film unter den Top-Treffern.
FA-16 (Kann)	Optionaler Filter im UI: Genre-Dropdown, Erscheinungsjahr-Range, Laufzeit-Range. Filterung clientseitig auf bereits geladenen Trefferlisten.	Bei aktiviertem Filter „Genre = Komödie“ erscheinen ausschließlich passende Titel.
FA-17 (Kann)	Optionale Auto-Vervollständigung: nach 300 ms Tastatur-Pause Anfrage an /api/movies/search mit den ersten Treffern als Dropdown.	Beim Tippen von „har“ erscheinen Vorschläge wie „Harry Potter ...“.

4.7 Optionale Funktionen

Aus dem Katalog der optionalen Anforderungen (OA-01 bis OA-08) wird in der ersten Iteration ausschließlich OA-05 (Medienanzeige: Poster/Thumbnails) bewusst eingeplant, weil sie das Nutzererlebnis spürbar aufwertet und TMDb die Poster-URLs ohnehin mitliefert. Alle übrigen optionalen Funktionen werden als Backlog für eine mögliche Folgeiteration dokumentiert und nicht zugesichert.

ID	Status in dieser Iteration	Begründung
OA-01 Statistiken	Backlog	Mittlerer Aufwand, kein Mehrwert für Kernanwendung.
OA-02 Bewertungen	Backlog	Erfordert zusätzliche Tabelle und UI-Komponenten.
OA-03 Empfehlungen	Backlog	TMDb liefert Ähnlichkeiten, UI-Aufwand ist aber relevant.
OA-04 Entdecken-Bereich	Backlog	Nicht kritisch; verlangt zusätzliches Routing/UI.
OA-05 Medienanzeige (Poster)	Eingeplant	Geringer Aufwand, hoher visueller Mehrwert. Erfüllt zugleich den Wunsch nach grafischer UI.
OA-06 SSO / Anzeigename	Backlog	Erfordert OAuth-Integration – außerhalb des Zeitbudgets.
OA-07 Lieblingsschauspieler	Backlog	Erfordert zusätzliche Entität, ggf. Periodischer Job.
OA-08 Präferenzalgorithmus	Backlog	Setzt OA-02 voraus; nicht im Zeitbudget.

4.8 Risikofaktoren

Risiko	Umgang
TMDb ist nicht verfügbar	Bei Filmanfrage wird eine Fehlermeldung zurückgegeben. Bei Listenabfrage wird die lokale DB angefragt.
Lokale DB ist nicht verfügbar	Fehlermeldung wird zurückgegeben.
Zu hohe Anfragelast	Anfragen werden in einer Queue gespeichert und nach spezifischem Intervall ausgeführt.

5 Produktdaten

5.1 Entity-Relationship-Modell

Das Datenmodell folgt der gemeinsam abgestimmten ER-Struktur. Drei Hauptentitäten (User, FilmeSerien, Genre) sind über zwei Beziehungs- bzw. Nebenentitäten (UserFilmeSerien, FilmeSerienGenre) verknüpft. Die Nebenentität UserFilmeSerien trägt das Listenattribut listentyp; FilmeSerienGenre verbindet Inhalte mit Genres im n:m-Verhältnis.

Entity-Relationship-Diagramm (Chen-Notation)

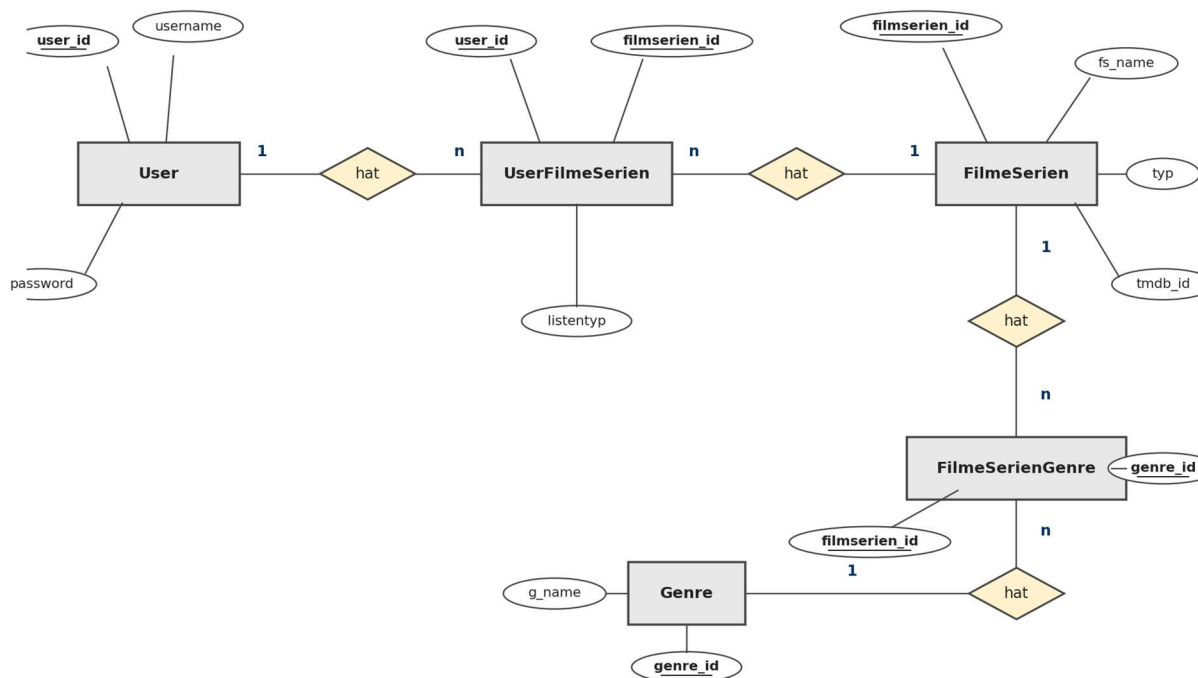


Abb. 2: Entity-Relationship-Diagramm (Chen-Notation)

5.2 Relationsschema

Aus dem ER-Modell ergibt sich folgendes Relationsschema:

Haupttabellen

- User { PK user_id : int, username : string, password : string }
- FilmeSerien { PK filmserien_id : int, fs_name : string, typ : string, tmdb_id : string }
- Genre { PK genre_id : int, g_name : string }

Nebentabellen

- UserFilmeSerien { PK,FK user_id : int, PK,FK filmserien_id : int, listentyp : string }
- FilmeSerienGenre { PK,FK filmserien_id : int, PK,FK genre_id : int }

5.3 Konkretisierung der Datentypen

Tabelle.Spalte	PostgreSQL	Nebenbedingung / Hinweis
User.user_id	SERIAL PRIMARY KEY	Künstlicher Schlüssel.
User.username	TEXT NOT NULL UNIQUE	Mindestlänge 3 (Validierung im Backend).
User.password	TEXT NOT NULL	bcrypt-Hash, nie Klartext.
FilmeSerien.filmserien_id	SERIAL PRIMARY KEY	Künstlicher Schlüssel.
FilmeSerien.fs_name	TEXT NOT NULL	Titel des Films/der Serie.
FilmeSerien.typ	TEXT NOT NULL	Werte: 'movie' 'tv'.
FilmeSerien.tmdb_id	TEXT	NULL bei lokal angelegten Einträgen (FA-10).
UserFilmeSerien.listentyp	TEXT NOT NULL	Werte: 'watchlist' 'watching' 'seen' 'local'.
Genre.g_name	TEXT NOT NULL UNIQUE	Bezeichner aus TMDb übernommen.

6 Produktleistungen (nicht-funktionale Anforderungen)

ID	Konkretisierung	Messung / Nachweis
NFA-01	Backend ist multithread-fähig (Flask im Threaded-Modus bzw. Werkzeug-Devserver mit threaded=True). Zielwert: 5 parallele Nutzer ohne spürbare Verzögerung.	Manueller Test mit 3 Browsern auf demselben Rechner während Abnahme.
NFA-02	Auslieferung ausschließlich als HTML/CSS/JS-Bundle vom Server; keine clientseitige Installation, kein Plugin nötig.	Aufruf im Browser ohne Vorinstallation funktioniert auf 2 unterschiedlichen Geräten.
NFA-03	Responsive Layout über CSS-Mediaqueries (Breakpoints bei 600 / 900 px). Mobile Darstellung wird als „nice to have“ behandelt.	Stichprobenartiger Test im Browser-DevTools-Mobilmodus auf Desktop.
NFA-04	Server-seitige Autorisierungsprüfung auf jeder Listen-Route (vgl. FA-04).	Identisch zu FA-04.
NFA-05	Passwort wird ausschließlich als bcrypt-Hash persistiert; keine Logs enthalten Klartext-Passwörter.	Stichprobe der User-Tabelle und der Logdateien zeigt keinen Klartext.
NFA-06	Antwortzeit < 2 s für typische Aktionen (Login, Suche, Liste anzeigen) bei lokaler Ausführung.	Browser-DevTools-Netzwerkprotokoll bei der Abnahme.

7 Qualitätsanforderungen

Qualitätsmerkmal (ISO 25010)	Zielniveau	Maßnahme
Funktionale Korrektheit	Hoch (Muss-Anforderungen 100 %)	Manuelle Testfälle pro FA-ID, kurz protokolliert.
Zuverlässigkeit	Mittel	Fallback-Mechanismus bei TMDb-Ausfall (vgl. Abschnitt 9.1).
Benutzbarkeit	Mittel–hoch	Konsistentes Layout, klare Fehlermeldungen, deutsche UI.
Sicherheit	Mittel	Passwort-Hashing, Sessions, server-seitige Autorisierung, keine SQL-Strings konkatenieren (Prepared Statements).
Wartbarkeit	Mittel	Klare Trennung Frontend/Backend, sprechende Namen, Commit-Konvention.
Portabilität	Hoch	Python + PostgreSQL ist plattformübergreifend lauffähig.

8 Benutzungsoberfläche (GUI)

Das Team hat den ausdrücklichen Wunsch formuliert, dass die Anwendung über eine grafische Oberfläche verfügt – im Sinne einer ansprechenden, postergestützten Web-UI und nicht einer reinen Tabellen- oder Konsolendarstellung. Dieser Punkt wird als optionales Qualitätsziel aufgenommen und durch folgende Mindeststandards untermauert:

- Klares Layout mit Kopfzeile (Logo links, Suchfeld mittig, Nutzermenü rechts).
- Hauptbereich als Kachelraster mit Postern (TMDb-Image-URL); Optional: Hover-/Tap-Effekt zeigt Titel.
- Detailseite mit großem Poster, Pflichtfeldern aus Lastenheft und Aktions-Buttons (Watchlist, Schon gesehen).
- Listenseiten (Watchlist, Schon gesehen, Schaue ich gerade) teilen sich eine Komponente und unterscheiden sich nur im Filter.
- Einsatz von Bootstrap 5 als CSS-Framework, um ohne JavaScript-Framework-Stack ein modernes Erscheinungsbild zu erreichen.
- Deutsche Beschriftungen, einheitliche Schriftarten (System-Stack).

Erweiterte Effekte (Animationen, Dark-Mode, drag-and-drop-Verschieben) werden ausdrücklich nicht zugesichert. Klickdummies bzw. Mockups werden – sofern Restbudget vorhanden – zu Beginn von Sprint 2 in Figma oder via HTML/CSS-Skizze erstellt.

9 Technische Produktumgebung

9.1 Architekturansatz

Die Anwendung ist als klassische dreischichtige Architektur konzipiert: Browser-Client, REST-Server, Datenhaltung. Der Server fungiert zugleich als Proxy zur externen TMDb-API. Bei Erreichbarkeit von TMDb werden die Daten von dort geladen und parallel in der lokalen Datenbank zwischengespeichert; bei Nicht-Erreichbarkeit greift der Server automatisch auf die lokale Datenbank zurück und liefert – falls vorhanden – einen reduzierten Datensatz mit einem entsprechenden Hinweis zurück. Ist auch der Cache (lokale DB) leer, wird HTTP 503 zurückgegeben.

Architektur – Sequenzdiagramm (Client / Server / TMDb / Lokale DB)

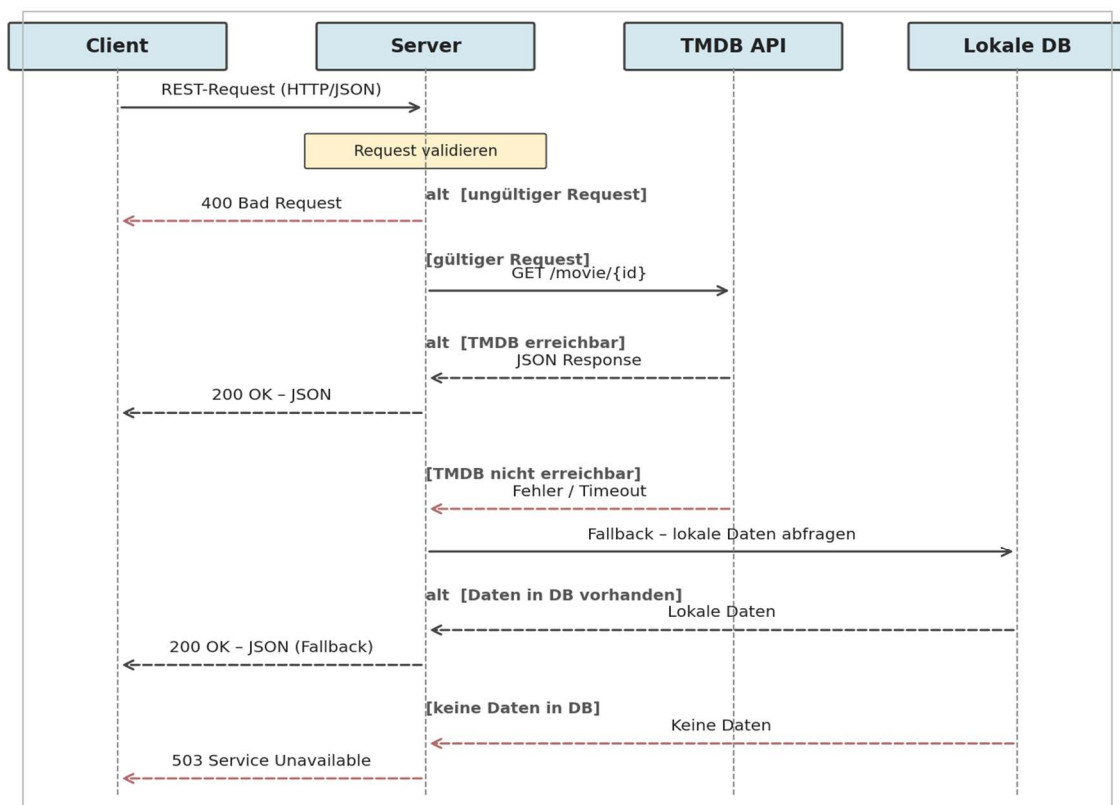


Abb. 3: Sequenzdiagramm – Kommunikation Client / Server / TMDb / Lokale DB

9.2 Empfohlener Technologiestack

Der Technologiestack wurde bewusst defensiv für ein Zweierteam mit einem Einsteiger ausgewählt. Alle Komponenten sind frei verfügbar, plattformunabhängig, gut dokumentiert und kommen ohne kostenpflichtige Drittkomponenten aus.

Schicht	Technologie	Begründung
Backend / API	Python 3.11+ mit Flask (oder FastAPI als Alternative)	Sehr lesbare Sprache, geringe Boilerplate-Last, große Community, ideal für Einsteiger; Flask hat eine kleine Lernkurve.
Datenbank	PostgreSQL	Zur Verfügung gestellter DB-Server der DHSN Glauchau.
DB-Zugriff	SQLAlchemy Core oder psycopg3-Modul	Prepared Statements out of the box, kein manuelles String-Konkatenieren.
Frontend	HTML5, CSS3, Vanilla JavaScript, Bootstrap 5	Vermeidet die hohe Lernkurve von React/Vue; das Frontend ist trotzdem responsive und visuell ansprechend.
HTTP-Client (Browser)	fetch-API	Im Browser integriert, keine zusätzliche Bibliothek.
TMDb-Anbindung	Pythons requests-Bibliothek	De-facto-Standard, einfache Syntax.
Passworthashing	bcrypt (passlib oder bcrypt-Modul)	Standardverfahren, saltet automatisch.
Versionierung	Git + GitHub-Repository (öffentlich oder privat)	Eine zentrale Quelle der Wahrheit; ermöglicht Pair-Programming und einfache Abgabe.
Entwicklungsumgebung	Visual Studio Code	Plattformübergreifend, gute Python-Unterstützung.

Hinweis zur Wahl Flask vs. Node.js/Express: Beide sind grundsätzlich geeignet. Die Entscheidung für Python beruht auf der erfahrungsgemäß geringeren Einstiegshürde (synchrone Programmierung, weniger Konfigurationsoverhead) und der besseren Eignung für den weniger erfahrenen Teampartner. Sollten beide Teammitglieder bereits JavaScript-Erfahrung haben, ist ein vollständiger Switch auf Node.js + Express vertretbar; ein Mischbetrieb ist nicht vorgesehen.

9.3 Externe Schnittstelle TMDb

- Verwendete API: TMDb v3 REST-API, base URL <https://api.themoviedb.org/3>.
- Authentifizierung: API-Key (kostenfrei, persönliches Entwicklerkonto), übertragen als Query-Parameter oder Bearer-Token.
- Verwendete Endpunkte (Minimalumfang): `/search/movie`, `/search/tv`, `/movie/{id}`, `/tv/{id}`, `/genre/movie/list`, `/genre/tv/list`.
- Rate-Limit-Strategie: Standardanfrage-Frequenz wird respektiert; bei HTTP 429 wird mit exponentiellem Backoff (1 s, 2 s, 4 s) erneut versucht und ggf. die lokale DB befragt.
- Datenschutz: Es werden keine personenbezogenen Daten an TMDb übertragen; ausschließlich Suchbegriffe und Film-IDs.

10 Programmabläufe (PAP)

Die folgenden Ablaufdiagramme zeigen die wichtigsten end-to-end-Abläufe der Anwendung. Sie konkretisieren die Architektur aus Abschnitt 9.1 für die zwei kritischsten Anwendungsfälle.

10.1 Anwendungsfall: Nutzer fragt einen Film an

Programmablaufplan - User fragt Film an

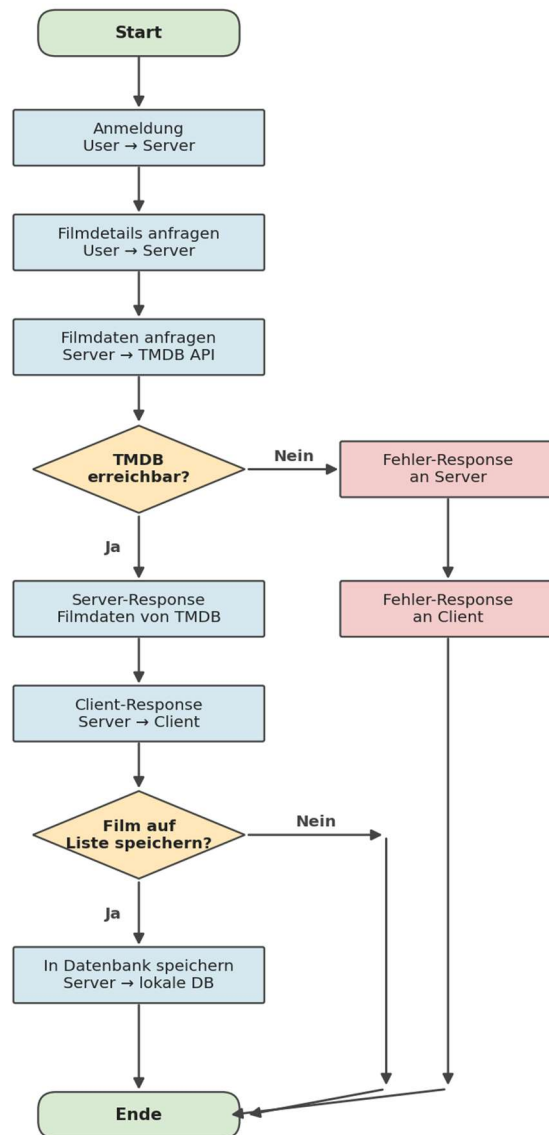


Abb. 4: Programmablaufplan – Anfrage eines einzelnen Films

10.2 Anwendungsfall: Nutzer ruft seine Liste auf

Programmablaufplan - User möchte seine Liste anschauen

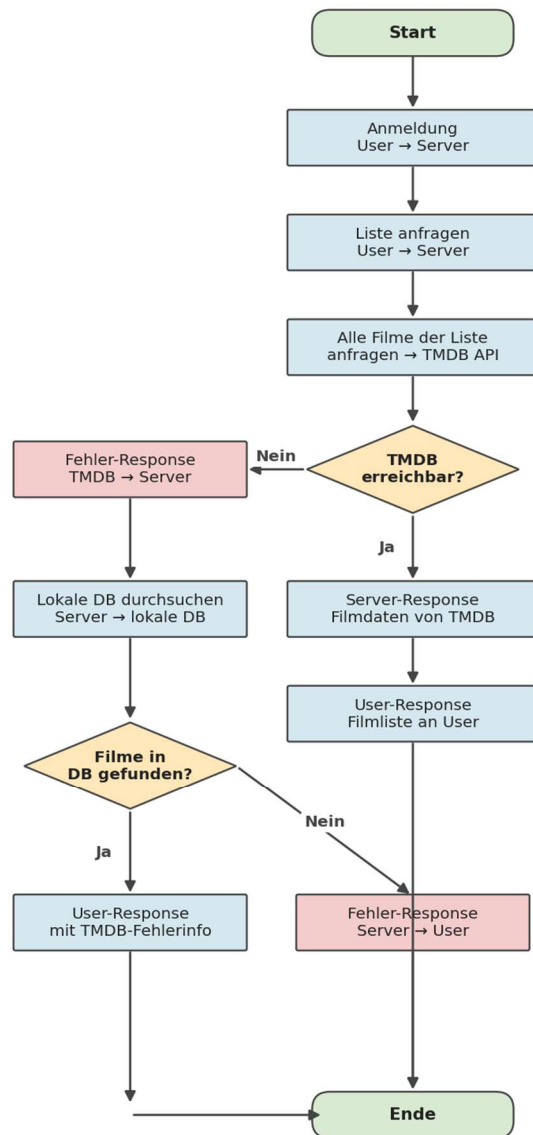


Abb. 5: Programmablaufplan – Aufruf einer Nutzerliste mit TMDB-Fallback

11 Iterationsplan und Aufwandsschätzung

11.1 Rahmen

Verfügbar sind 75 Lehrveranstaltungen à 45 Minuten je Teammitglied, was ca. 56 Stunden Projektzeit pro Person bzw. 112 Personenstunden im Gesamtteam entspricht. Diese Zahl ist eine Bruttogröße.

Erfahrungsgemäß gehen davon ca. 30 % für Einarbeitung, Recherche, Tooling, Bugfixing, Dokumentation und Abnahmevorbereitung verloren. Als effektive Entwicklungszeit kalkuliert das Team daher konservativ mit etwa 75 Personenstunden, was in vier Sprints à ca. 18–20 Personenstunden zerfällt.

11.2 Risikofaktoren

- Lernkurve des Einsteigers in Python/Flask und Git (Risiko: bis zu 10 h zusätzlicher Aufwand in Sprint 1).
- TMDb-API-Verständnis (Authentifizierung, JSON-Strukturen) – Mitigation: dedizierte Lernspike in Sprint 2.
- Krankheit / Klausurphase – Pufferzeit am Ende eingeplant.
- Unterschiedliche Verfügbarkeit beider Teammitglieder – fester Pair-Programming-Termin pro Woche.
- Drohendes Feature-Creep durch optionale Anforderungen – wird durch klares Backlog-Statement in 4.7 entschärft.

11.3 Sprintplan

Sprint	Inhalt	Aufwand (PT je 2 Personen)
Sprint 1 – Setup & Grundlagen	Git-Repository, README, Branch-Konventionen. Python-/Flask-Setup, Projektstruktur, lokale Datenbank. Datenbankschema laut Abschnitt 5 anlegen (DDL-Skript). Erste API-Route /api/healthcheck als „Hello World“. Tooling: linting, Test-Runner (pytest).	ca. 20 h
Sprint 2 – Auth & Katalog	Registrierung, Login, Logout (FA-01, FA-02, FA-05). TMDb-API-Anbindung, Such- und Detail-Endpunkte (FA-06–FA-09, FA-15). Caching-Strategie in lokaler DB. Einfache HTML-Seiten mit Bootstrap-Basisstruktur.	ca. 22 h
Sprint 3 – Listen-Logik	Watchlist, Schon-gesehen, Schaue-ich-aktuell (FA-11–FA-13). Verschieben zwischen Listen (FA-14). Lokale Nutzerliste (FA-10). Listen-UI mit Kachelraster und Poster (OA-05).	ca. 20 h
Sprint 4 – Polish, Tests & Abgabe	Fehlerbehandlung Frontend/Backend, konsistente Statusmeldungen. Responsive Anpassung (NFA-03). Manuelle Testprotokolle, Bugfixing. Dokumentation, Demovideo, Abgabe.	ca. 13 h (inkl. ~5 h Puffer)

11.4 Aufgabenverteilung

Pair-Programming ist für die ersten beiden Sprints fest eingeplant. In Sprint 3 und 4 erfolgt eine Aufteilung in Backend-/Frontend-Schwerpunkte, ohne dass eine Person ein Modul allein verantwortet (Bus-Faktor 2).

- Schwerpunkt fortgeschrittenes Teammitglied: Architektur, TMDb-Anbindung, Datenbank, Listen-Logik im Backend.
- Schwerpunkt Einsteiger: HTML-Templates, Bootstrap-Styling, manuelle Tests, README/Dokumentation.
- Gemeinsam: Sprint-Planung am Anfang jedes Sprints, Demonstration am Ende, Codereviews per Pull-Request.

12 Abnahmekriterien

Die Anwendung gilt als abgenommen, wenn die folgenden Kriterien in einer Live-Demonstration erfüllt sind. Die Reihenfolge entspricht der vorgesehenen Demo-Reihenfolge.

- Erfolgreiche Registrierung eines neuen Nutzers und Login.
- Erfolgreicher Login eines bestehenden, vom Prüfer benannten Demo-Nutzers.
- Suche nach einem zufällig genannten Filmtitel; Detailansicht öffnet sich mit allen Pflichtfeldern.
- Hinzufügen des Films auf die Schon-gesehen-Liste; der Eintrag erscheint dort.
- Wechsel zu einem zweiten Konto: dort ist der Eintrag nicht sichtbar (FA-04-Nachweis).
- Demonstration des Fallback-Pfads durch temporäres Abschalten der TMDb-Verbindung (Netzwerk trennen oder API-Key falsch setzen) und erneuter Abruf einer bereits vorhandenen Liste.
- Visuelle Inspektion: Poster werden angezeigt, Layout ist auf einem Desktop-Browser ohne offensichtliche Brüche.

13 Anhänge und mitgeltende Dokumente

Dokument	Inhalt	Bezug
Lastenheft v1.4 (19.05.2026)	Auftraggeber-Anforderungen, FA-/NFA-/OA-IDs	Grundlage des Pflichtenhefts
Architekturansatz.md	Sequenzdiagramm (Mermaid) Client/Server/TMDb/DB	Eingebettet als Abb. 3
Programmablaufplan.md	PAP für Filmanfrage und Listenabfrage (Mermaid)	Eingebettet als Abb. 4 und Abb. 5
Relationsschema.md	Tabellen, Schlüssel, Beziehungen in Textform	Übernommen in Abschnitt 5.2
erd_user_listen.drawio	ER-Diagramm (Chen-Notation) in draw.io	Visualisiert als Abb. 2
README.md	Kurzüberblick, Team, Grundidee	Hintergrundinformation

14 Glossar

Begriff	Erläuterung
Watchlist	Liste der Inhalte, die ein Nutzer noch sehen möchte.
Schon-gesehen-Liste	Liste bereits konsumierter Inhalte.
Schaue-ich-aktuell-Liste	Inhalte, die ein Nutzer aktiv verfolgt.
Lokale Nutzerliste	Nutzerspezifische Einträge, die nicht im zentralen Katalog vorhanden sind (FA-10).
TMDb	The Movie Database – externe, frei nutzbare Film- und Serien-API.
PWA	Progressive Web App – Web-Anwendung mit App-ähnlichem Verhalten auf Mobilgeräten.
SSO	Single Sign-On – Anmeldung über externen Identitätsanbieter.
REST	Representational State Transfer – Architekturprinzip für HTTP-APIs.
bcrypt	Passwort-Hashing-Verfahren mit eingebautem Salting und konfigurierbarem Kostenfaktor.
Sprint	Zeitlich begrenzte Iteration mit festgelegtem Sprintziel (hier ca. 2 Wochen / 20 h pro Team).
Lokale DB	Eine Datenbank, die in Verbindung mit dem zur Verfügung gestellten Server steht und nicht mit dem Client.